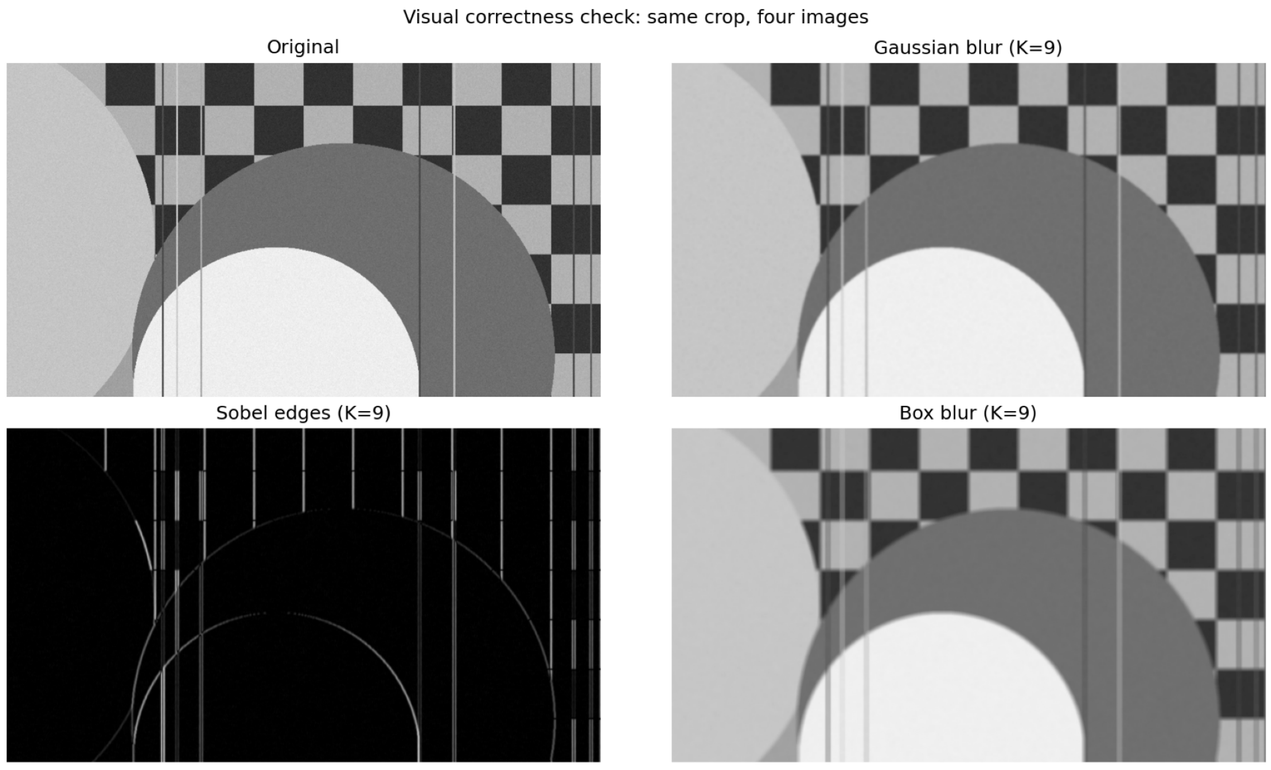
Shared-Memory, GPU and Distributed Parallelisation of 2D Image Convolution

Saatvik Das · Saket Rama · Zaid Khan

Manipal Institute of Technology, Bengaluru · Guide: Prof. Anil Kumar · PCAP, Semester V



One image filter.
Five ways to run it.

Serial
one CPU core, the baseline

OpenMP
16 CPU threads

CUDA naive
GPU, plain memory

CUDA constant
filter in fast constant memory

CUDA tiled
image tiles in shared memory

Original, Gaussian blur, edge detection and box blur. Every version produces the same picture.

8.5×
faster with 16 CPU threads

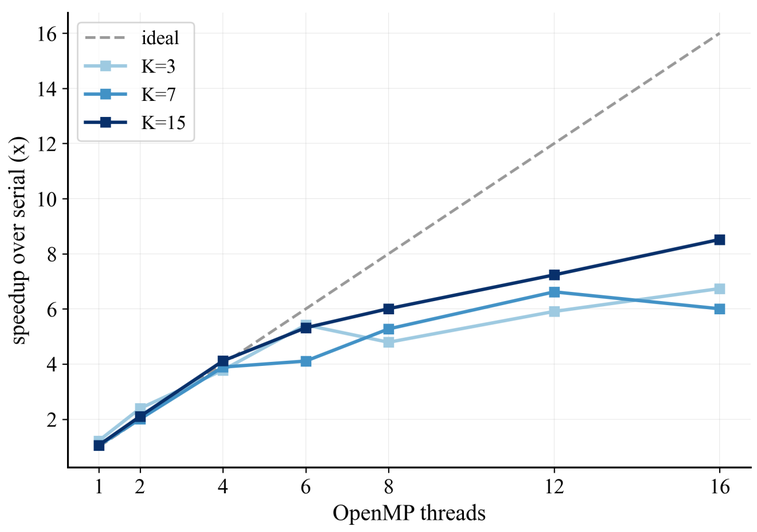
389×
faster on the GPU, filter only

104×
faster on the GPU, whole job

70–93%
of GPU time is copying data

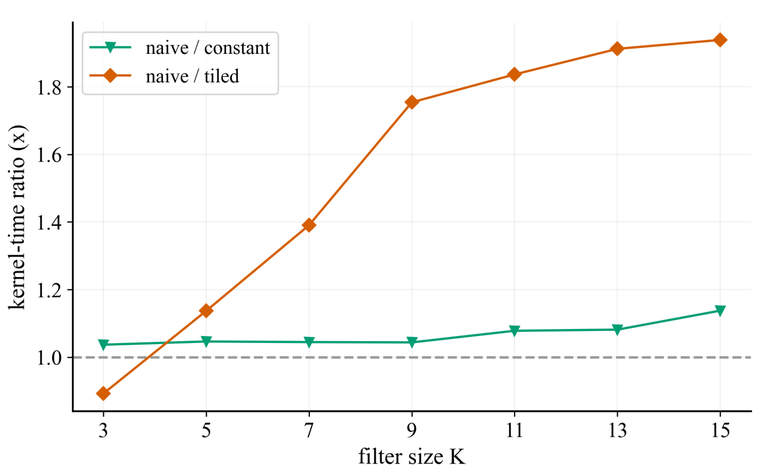
728 runs on 4K and 8K images, filters 3×3 to 15×15. Every output verified against the serial result.

CPU threads stop helping



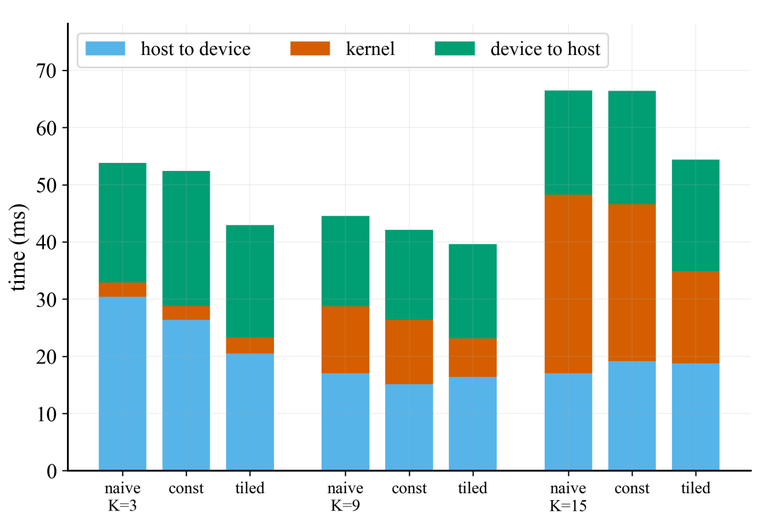
Gains flatten after about 4 threads.
Memory, not cores, is the limit.

Tiling helps big filters



Shared-memory tiles run 1.9× faster at 15×15, but not at 3×3.

Copying eats the GPU's lead



Moving the image to and from the GPU takes most of the time.

What we learned

- **Memory, not core count,** decides how fast the filter runs.
- The GPU's caches already do most of the reuse, so tiling helps **only for large filters**.
- Always count the copying: **389× becomes 104×** for the whole job.

Code, results and full report
github.com/PCAP-ZSS/ConvoX · i7-12650H, RTX 3050 Laptop GPU, CUDA 12.6